AWS Security Hub Comprehensive Demo Runbook

This runbook provides a detailed, step-by-step guide for demonstrating AWS Security Hub with integrations to GuardDuty, Inspector, Macie, and Config. It includes manual setup, multi-service integration, auto-remediation workflows, testing, verification, and cleanup. Designed for hands-on use in the AWS Management Console within a sandbox account (recommend us-east-1 for broadest service availability). Assumes familiarity with basic AWS concepts. Current as of March 2026.

Prerequisites

- **AWS Account**: Use a dedicated sandbox account with AdministratorAccess IAM policy attached to your user or role. Avoid production environments to prevent unintended changes.
- **Required IAM Policies**: Attach these managed policies to your IAM user/role:
  - SecurityHubFullAccess
  - IAMFullAccess (for creating roles and policies)
  - LambdaFullAccess
  - EventsFullAccess (for EventBridge)
  - SSMFullAccess (for Systems Manager patching)
- **Service Enablement**:
  - Enable AWS CloudTrail in us-east-1 (multi-region trail recommended) for auditing and EventBridge integration.
  - Enable AWS Config in us-east-1, configured to record all supported global and regional resources.
- **Region**: Perform all steps in us-east-1 unless specified.
- **Time Estimate**: 2-3 hours, including wait times for findings generation and sync (e.g., 15-30 minutes for initial integrations).
- **Best Practices**: Use AWS Free Tier eligible resources where possible (e.g., t3.micro instances). Monitor costs via AWS Cost Explorer. Ensure multi-factor authentication (MFA) is enabled on your account.

Step 1: Enable Security Hub and Core Integrations

1. Navigate to the AWS Management Console > Services > Security Hub.
2. If not already enabled, click **Enable Security Hub**. Accept defaults for standards (e.g., AWS Foundational Security Best Practices, CIS AWS Foundations Benchmark).
3. Go to **Integrations** tab.
4. Enable the following integrations one by one:
   - Amazon GuardDuty: Monitors for malicious activity.
   - Amazon Inspector: Scans for vulnerabilities.
   - Amazon Macie: Discovers sensitive data.

    - AWS Config: Evaluates resource configurations.
5. Ensure AWS Config is actively recording resources (check Config > Settings).
6. **Verification**:
    - Refresh the Integrations page after 15-30 minutes. All should show as "Enabled".
    - If issues arise (e.g., sync delays), check AWS Health Dashboard for service disruptions.
    - Initial baseline findings may appear; note them for comparison later.

Step 2: Generate Test Findings

Create controlled security issues to simulate real-world findings. This triggers detections across services.

- For GuardDuty:
    - Launch a t3.micro EC2 instance (Amazon Linux 2 AMI).
    - From the instance, run aws s3 ls//<bucket name> --no-sign-request to simulate unauthorized access.
    - Expected finding type: Backdoor:EC2/C&CActivity.B!DNS or similar threat detection. Wait time: 5-15 minutes.
- For Inspector:
    - On the same EC2 instance, run sudo yum update -y but interrupt midway (or skip). Reboot if needed.
    - Ensure Inspector scans are enabled.
    - Expected finding type: CVE vulnerabilities on unpatched packages (e.g., high-severity CVEs).
    - Wait time: 10-20 minutes (scan initiation).
- For Macie:
    - Create an S3 bucket named macie-test-2026 (public access optional for demo).
    - Upload a file test-pii.txt containing sample PII: "SSN: 123-45-6789".
    - Run a Macie sensitive data discovery job on the bucket.
    - Expected finding type: SensitiveData:S3Object/Multiple (PII detection).
    - Wait time: 5-10 minutes.
- For Config:
    - In Config > Rules, add built-in rule "EC2_INSTANCE_NO_PUBLIC_IP".
    - Launch an EC2 instance with a public IP.
    - Expected finding type: NON_COMPLIANT (instance has public IP).
    - Wait time: Immediate.

Verification:

- Go to Security Hub > Findings.
- Filter by ProductName (e.g., GuardDuty, Inspector).
- Expect 4+ findings. Note resource ARNs (e.g., EC2 instance ID, S3 bucket ARN) for later cross-referencing.
- Tip: Use filters like SeverityLabel: HIGH to prioritize.

Step 3: Setup Auto-Remediation

Automate fixes using Lambda, Security Hub custom actions, EventBridge, and automation rules. This demonstrates serverless remediation.

3.1 Create Lambda Functions

Navigate to Lambda console > Functions > Create function (Author from scratch, Python 3.12 runtime, 128 MB memory, 5-minute timeout). Create three functions with the code below. Add error handling for robustness.

- **Lambda 1: S3Encrypt (for Macie/Config findings)**

```
import json
import boto3

def lambda_handler(event, context):
    sh = boto3.client('securityhub')
    s3 = boto3.client('s3')
    try:
        finding = event['detail']['findings'][0]
        bucket = finding['Resources'][0]['Id'].split(':::')[-1]
        s3.put_bucket_encryption(
            Bucket=bucket,
            ServerSideEncryptionConfiguration={
                'Rules': [{'ApplyServerSideEncryptionByDefault':
{'SSEAlgorithm': 'AES256'}}]
            }
        )
        sh.batch_update_findings(
            FindingIdentifiers=[{'Id': finding['Id'], 'ProductArn':
finding['ProductArn']}],
            Workflow={'Status': 'RESOLVED'},
```

```
            Note={'Text': 'Auto-remediated: S3 encryption enabled'}
        )
        return {'status': 'Success'}
    except Exception as e:
        print(f"Error: {str(e)}")
        return {'status': 'Failure'}
```

- **Lambda 2: ECPatch (for Inspector findings)**

```
import json
import boto3

def lambda_handler(event, context):
    ssm = boto3.client('ssm')
    try:
        finding = event['detail']['findings'][0]
        instance_id = finding['Resources'][0]['Id'].split('/')[-1]
        ssm.send_command(
            InstanceIds=[instance_id],
            DocumentName='AWS-RunPatchBaseline',
            Parameters={'Operation': ['Install'], 'RebootIfNeeded': ['true']}
        )
        return {'status': 'Success'}
    except Exception as e:
        print(f"Error: {str(e)}")
        return {'status': 'Failure'}
```

- **Lambda 3: SGHarden (for GuardDuty/Config findings)**

```
import json
import boto3

def lambda_handler(event, context):
    ec2 = boto3.client('ec2')
    try:
        finding = event['detail']['findings'][0]
```

```
        sg_id =
finding['Resources'][0]['Details']['AwsEc2SecurityGroup']['GroupId']
        ec2.revoke_security_group_ingress(
            GroupId=sg_id,
            IpPermissions=[
                {'IpProtocol': 'tcp', 'FromPort': 22, 'ToPort': 22,
'IpRanges': [{'CidrIp': '0.0.0.0/0'}]},
                {'IpProtocol': 'tcp', 'FromPort': 3389, 'ToPort': 3389,
'IpRanges': [{'CidrIp': '0.0.0.0/0'}]}
            ]
        )
        return {'status': 'Success'}
    except Exception as e:
        print(f"Error: {str(e)}")
        return {'status': 'Failure'}
```

Permissions: For each Lambda, create an execution role with the following inline policy (attach via IAM > Roles):

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": [
                "s3:PutBucketEncryption",
                "ssm:SendCommand",
                "ec2:RevokeSecurityGroupIngress",
                "securityhub:BatchUpdateFindings"
            ],
            "Resource": "*"
        },
        {
            "Effect": "Allow",
            "Action": "logs:CreateLogGroup",
            "Resource": "arn:aws:logs:*:*:*"
```

```
        },
        {
            "Effect": "Allow",
            "Action": [
                "logs:CreateLogStream",
                "logs:PutLogEvents"
            ],
            "Resource": "arn:aws:logs:*:*:log-group:/aws/lambda/*:*"
        }
    ]
}
```

3.2 Create Security Hub Custom Actions

Security Hub CSPM> Custom actions > Create action (three total):

- **AutoEncryptS3**: Description: "Remediate S3 encryption". Target: Lambda1 ARN. Custom action ID: remediateS3.
- **AutoPatchEC2**: Description: "Patch EC2 instances". Target: Lambda2 ARN. ID: remediateEC2.
- **CloseOpenSG**: Description: "Harden security groups". Target: Lambda3 ARN. ID: remediateSG.

3.3 Create EventBridge Rules

EventBridge > Rules > Create rule (three total, one per Lambda):

- **Pattern** (common to all):

```
{
    "source": ["aws.securityhub"],
    "detail-type": ["Security Hub Findings - Imported"],
    "detail": {
        "findings": {
            "Severity": {
                "Label": ["HIGH", "CRITICAL"]
            }
        }
    }
```

```
}
```

- **Target**: The corresponding Lambda function.
- **Additional Filters**: Customize per rule (e.g., for S3: add findings.ProductName: ["Macie"]).
- Tip: Test rules with EventBridge > Event buses > Send events (simulate Security Hub events).

3.4 Create Security Hub Automation Rules

Security Hub > Automation rules > Create rule (three total):

- Rule Name: S3 Auto. Criteria: ProductName: MACIE, SeverityLabel: HIGH. Action: Custom action: AutoEncryptS3.
- Rule Name: EC2 Auto. Criteria: ProductName: INSPECTOR, ResourceType: AwsEc2Instance. Action: Custom action: AutoPatchEC2.
- Rule Name: SG Auto. Criteria: ProductName: CONFIG OR GUARD DUTY, Title CONTAINS "Security Group". Action: Custom action: CloseOpenSG.

Enable rules after creation.

Step 4: Test Full Automation

1. Regenerate findings from Step 2.
2. Wait 10-20 minutes for detections and triggers.
3. **Verification**:
    - Security Hub > Findings: Filter by WorkflowStatus: RESOLVED. Confirm remediation notes.
    - Lambda > Monitoring: Check invocations and logs for successes/errors.
    - Dashboard: Security score should improve (e.g., from ~78% to 96%).
    - Resource Checks:
        - S3: aws s3api get-bucket-encryption --bucket macie-test-2026 shows AES256.
        - EC2: Use SSM > Instance information to confirm patches.
        - SG: Describe security group; no open 22/3389 from 0.0.0.0/0.

Complete Test Matrix:

- Test Scenario: GuardDuty Threat. Manual Check: Simulate EC2 exfil (S3 ls). Auto Outcome: SG ports closed. Pass Criteria: No open 22/3389 (ec2 describe-security-groups).
- Test Scenario: Inspector CVE. Manual Check: Unpatched EC2. Auto Outcome: SSM patch baseline run. Pass Criteria: Patches applied, instance rebooted if needed.
- Test Scenario: Macie PII. Manual Check: S3 bucket with SSN. Auto Outcome: AES256 encryption enabled. Pass Criteria: get-bucket-encryption confirms SSE.

- Test Scenario: Config Public IP. Manual Check: Launch public EC2. Auto Outcome: Compliance restored (manual stop/start to private). Pass Criteria: Rule status: COMPLIANT.
- Test Scenario: Score Delta. Manual Check: Pre: ~80%. Auto Outcome: Post: >95%. Pass Criteria: Dashboard confirmation.

Step 5: Reporting and PDF Export

1. **Export Findings**: Security Hub > Findings > Select all > Actions > Export to CSV (include resolved ones with notes). Save as remediated-findings.csv.
2. **Screenshots**: Capture before/after:
    - Dashboard scores.
    - Lambda invocation logs.
    - Resource configurations (e.g., S3 encryption, EC2 patch status).
3. **Generate PDF Report**:
    - **Option 1: CLI with AWS CLI and Pandoc (Local Machine)**: Install Pandoc and wkhtmltopdf if needed.

      Bash

      ```bash
      # Export insights or findings
      aws securityhub get-findings --filters
      'WorkflowStatus=[{"Comparison":"EQUALS","Value":"RESOLVED"}]' --
      output json > findings.json
      # Convert to PDF (add CSS for styling if desired)
      pandoc findings.json -o SecurityHub-Demo-Report.pdf --pdf-
      engine=wkhtmltopdf --variable version=1.0
      ```

      Enhance with markdown: Create a report.md summarizing steps, paste findings, then pandoc report.md -o report.pdf.

    - **Option 2: Console Print to PDF**: Navigate to Security Hub > Findings/Insights/Dashboard. Use browser Print > Save as PDF (Chrome/Firefox recommended for full-page capture). Combine pages using tools like Adobe Acrobat or online mergers.
    - **Option 3: Automated via Code (Advanced)**: Use Python with libraries like reportlab (if available in your env) or export to HTML then convert. For demo, stick to CLI for simplicity.
4. **Best Practices**: Include timestamps, account ID, and remediation evidence in the PDF for audit trails.

Cleanup (Mandatory to Avoid Costs)

1. **Security Hub**: Settings > General > Suspend security standards; disable all integrations.
2. **Delete Resources**:
    - EC2 instances and S3 buckets created for testing.
    - Lambda functions.
    - EventBridge rules.
    - Security Hub custom actions and automation rules.
3. **IAM Cleanup**: Detach or delete temporary policies/roles.
4. **Other Services**: Stop AWS Config recording and CloudTrail logging if not needed.
5. **Verification**: Check Billing > Cost Explorer for zero activity post-cleanup. Use AWS Nuke or similar tools for thorough sandbox reset if desired.